Completely-Fun Version 3

Subject: **Analysis & Design Algorithms**

Instructors: **Dr.Bahaa & Eng.Sama**

Project: **Music Playlist Generator with ChatBot**

Worked On

Name: Noran Ahmed **ID:** 42410001

Name: Yousef Mohamed **ID:** 42410515

Name: Kerolos Nady **ID:** 42410542

Name: Shrouk Khalaf **ID:** 42410575

Name: Philopateer Waheed **ID:** 42410086

Name: Merna Ashraf **ID:** 42410342

Completely-Fun Report

This report explains the implementation of the new YouTube Music Playlist Manager & ChatBot. project and analyzes the Data Structures and Algorithms used based on Time Complexity, CPU Effort, Memory Usage, Test Cases, Outputs, and Algorithm Selection.

1. Project Overview

The project is a console-based music playlist management system developed using C++. It allows the user to add, remove, display, play, search, sort, save songs, and interact with a simple ChatBot.
The new implementation is simpler than the old version because it mainly uses STL containers instead of manually implemented linked lists, stacks, and queues.

2. Main Features

- Add songs with title, artist, genre, duration, and YouTube link.
- Validate YouTube links before inserting songs.
- Store songs dynamically using vector<Song>.
- Play songs using queue<Song> in FIFO order.
- Undo the last removal using stack<Song> in LIFO order.
- Sort songs alphabetically by title using Quick Sort.
- Search songs by title using Binary Search after sorting.
- Save playlist data and activity logs into a CSV file.
- Use a simple ChatBot for basic commands.
-

3. General Idea of the Project

The project simulates a music playlist manager. The user interacts with a menu and selects the required operation. Each song is represented as an object of the Song structure, and all songs are stored inside a vector in the PlaylistManager class.

1. When a song is added, the program validates the YouTube link.
2. If the link is valid, the song is inserted into the playlist vector.
3. The same song is pushed into the play queue.
4. The action is recorded in the logs vector.
5. When searching, the playlist is sorted first using Quick Sort, then Binary Search is applied.
6. When removing, the last song is saved in a stack so it can be restored using Undo.

4. Classes and Structures Explanation

4.1 Song Structure

The Song structure represents a single song in the playlist.

Attribute	Description
title	Stores the song title

artist	Stores the artist name
genre	Stores the music genre
youtubeLink	Stores the YouTube URL
duration	Stores duration in seconds

Main Functions in Song

getFormattedDuration(): Converts duration from seconds into minutes:seconds format. Example: 250 seconds becomes 4:10.
display(): Prints the song details including title, artist, genre, formatted duration, and YouTube link.

4.2 PlaylistManager Class

PlaylistManager is the core class responsible for managing all playlist operations.

Member	Type	Purpose
name	string	Stores playlist name
songs	vector<Song>	Stores all songs
logs	vector<string>	Stores activity history
playQueue	queue<Song>	Stores songs waiting to be played
undoStack	stack<Song>	Stores removed songs for undo

Function	Purpose	Time Complexity	Space Complexity
validYoutube()	Checks whether the URL contains youtube.com or youtu.be.	O(m), where m is URL length	O(1)
logActivity()	Adds a timestamped activity entry to the logs vector.	O(1) average	O(1)
addSong()	Validates and adds a song to the playlist and play queue.	O(1) average, O(n) if vector resizes	O(1)
removeLastSong()	Removes the last song and stores it in undoStack.	O(1)	O(1)
displayAll()	Displays all songs in the playlist.	O(n)	O(1)
playNext()	Plays and removes the front song from the queue.	O(1)	O(1)
undoRemove()	Restores the last removed song from the stack.	O(1) average	O(1)
searchSong()	Sorts songs then uses Binary Search to find a title.	O(n log n) average, $O(n^2)$ worst	O(log n) average

sortPlaylist()	Sorts the playlist alphabetically using Quick Sort.	O(n log n) average, $O(n^2)$ worst	O(log n) average
saveCSV()	Writes songs and logs into a CSV file.	O(n + l)	O(1)

4.3 ChatBot Class

The ChatBot class provides a simple text-based interface. It responds to hello, help, and bye commands. Any other input produces the output: I am listening...

Command	Output
hello	Hello User
help	Commands: add, remove, play, search
bye	Goodbye
Any other input	I am listening...

5. Data Structures Used

5.1 Vector

The new code uses vector<Song> instead of a manually implemented Doubly Linked List. Vector was chosen because it supports fast indexing, works efficiently with Quick Sort and Binary Search, and is simple to manage.

Feature	Vector	Doubly Linked List
Random Access	O(1)	O(n)
Insert at End	O(1) average	O(1)
Delete Last	O(1)	O(1)
Sorting	Easy and efficient	Harder
Binary Search	Suitable	Not suitable
Memory Usage	Lower overhead	Higher overhead due to pointers

5.2 Queue

queue<Song> is used for the Play Next feature because music playback follows FIFO order: the first song added should be the first song played.

5.3 Stack

stack<Song> is used for Undo because undo operations follow LIFO order: the last removed song should be the first restored.

5.4 Vector Logs

vector<string> stores activity logs such as adding and removing songs with timestamps.

6. Algorithms Used

6.1 Quick Sort

Quick Sort is used to sort the playlist alphabetically by song title. The algorithm selects the last element as pivot, partitions the vector, and recursively sorts the left and right parts.

Case	Time Complexity
Best	O(n log n)
Average	O(n log n)
Worst	$O(n^2)$

Quick Sort was chosen because it is fast on average, suitable for vectors,
and demonstrates divide-and-conquer, recursion, pivot selection, and partitioning.

6.2 Binary Search

Binary Search is used in searchSong() after sorting the playlist. It repeatedly checks the middle element and eliminates half of the remaining search range.
Binary Search itself is O(log n), but because searchSong() sorts before searching, the full operation is O(n log n) on average and $O(n^2)$ in the worst case due to Quick Sort worst-case behavior.

7. Comparison Between Existing Algorithms and Better Alternatives

Algorithm	Best	Average	Worst	Memory	Best Use
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n) average	Fast general sorting
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Guaranteed performance
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Small or nearly sorted data
std::sort	O(n log n) average	O(n log n)	O(n log n) typical	O(log n)	Best practical C++ option

The current code uses Quick Sort because it is clear academically and efficient on average. However, in production-level C++ code, std::sort would usually be the better practical option because it is highly optimized.

Search Algorithm	Requires Sorted Data?	Best	Worst	Use Case
Linear Search	No	O(1)	O(n)	Small or unsorted playlist
Binary Search	Yes	O(1)	O(log n)	Large sorted playlist

Current searchSong()	Sorts first	O(n log n)	$O(n^2)$	Current implementation

Binary Search was chosen because it is much faster than Linear Search on sorted data. However, sorting before every search is inefficient when many searches are performed. A better design would keep an isSorted flag and sort only when needed.

8. Complexity Analysis

Operation	Data Structure / Algorithm	Time Complexity	Space Complexity
Add Song	Vector + Queue	O(1) average	O(1)
Remove Last	Vector + Stack	O(1)	O(1)
Undo Remove	Stack + Vector	O(1) average	O(1)
Display All	Vector Traversal	O(n)	O(1)
Play Next	Queue	O(1)	O(1)
Sort Playlist	Quick Sort	O(n log n) average, $O(n^2)$ worst	O(log n) average
Search Song	Quick Sort + Binary Search	O(n log n) average	O(log n)
Save CSV	File Writing	O(n + l)	O(1)
ChatBot Input	String Processing	O(k)	O(1)

Data Structure	Operation	Complexity
Vector	Access by index	O(1)
Vector	Insert at end	O(1) average
Vector	Remove from end	O(1)
Vector	Traverse	O(n)
Queue	Push / Pop	O(1)
Stack	Push / Pop	O(1)
Vector Logs	Add log	O(1) average

9. Test Cases

Test Case	Input / Action	Expected Output	Result
1	Add valid song	Song Added	Song is added to songs, playQueue, and logs
2	Add invalid YouTube link	Invalid YouTube Link	Song is rejected
3	Display empty playlist	Empty Playlist	No songs displayed
4	Remove from empty playlist	Empty Playlist	No removal occurs

5	Remove last song	Removed Last Song	Song is moved to undoStack
6	Undo after removal	Undo Done	Song is restored
7	Play next after adding song	Now Playing: song title	Front queue song is played
8	Search existing title	Found Song	Song details displayed
9	Search missing title	Song Not Found	No song displayed
10	Save CSV	Saved Successfully	FunPlaylist.csv is created

10. Why These Algorithms and Data Structures Were Chosen

10.1 Why Vector Instead of Linked List?

Vector was selected because the new code depends heavily on index-based access, Quick Sort, and Binary Search. Binary Search needs O(1) access to the middle element, which vector provides but linked list does not.

10.2 Why Queue for Play Next?

Queue was selected because playlist playback follows FIFO order: first song added is the first song played.

10.3 Why Stack for Undo?

Stack was selected because undo operations follow LIFO order: the last removed song should be restored first.

10.4 Why Quick Sort?

Quick Sort was selected because it is fast on average, suitable for vector indexing, and useful for demonstrating divide-and-conquer. Its weakness is the possible $O(n^2)$ worst case when pivot choice is poor.

10.5 Why Binary Search?

Binary Search was selected because it reduces searching from O(n) to O(log n) after sorting. It is appropriate because songs are stored in a vector.

11. Limitations of the New Code

- Search sorts the playlist every time.
- Quick Sort uses the last element as pivot, which may cause $O(n^2)$ worst-case performance.
- CSV saving does not wrap fields in quotes, so commas inside titles or artist names may break the CSV format.
- Undo does not log restored songs.
- There is no load CSV feature.
- There is no edit song feature.

- Search is only by title.
- YouTube validation is basic and case-sensitive.
- The ChatBot is simple and not connected to playlist operations.

12. Conclusion

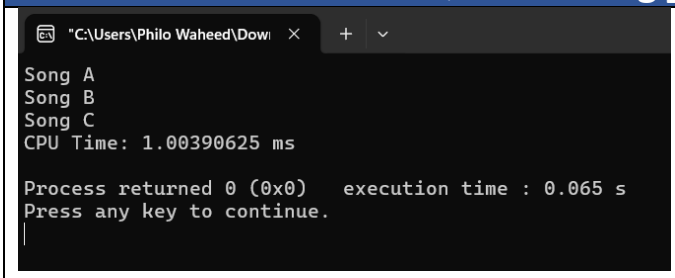
The new YouTube Music Playlist Manager successfully demonstrates important Data Structures and Algorithms in a simpler and cleaner way.

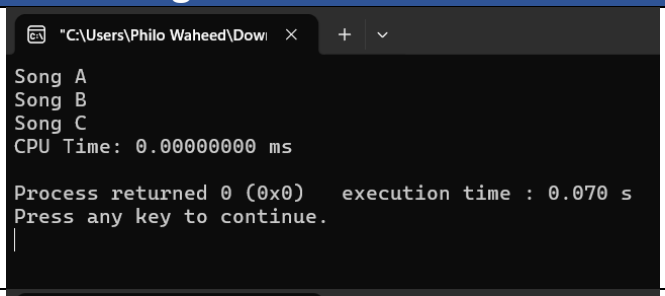
- vector for playlist storage.
- queue for playing songs in order.
- stack for undo operations.
- Quick Sort for sorting songs by title.
- Binary Search for efficient title searching.
- CSV file handling for saving data.
- A simple ChatBot for user interaction.

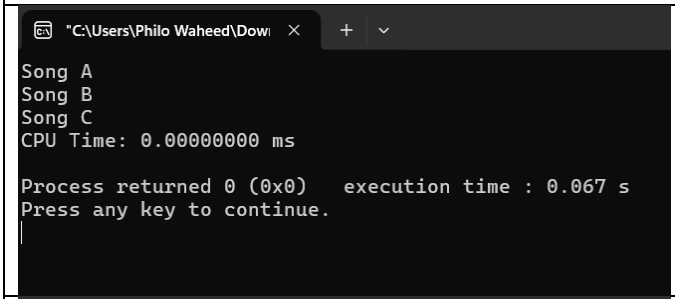
The main advantage of the new version is simplicity and better compatibility with Binary Search due to the use of vector. However, the old version was more advanced because it included more algorithms and features such as Merge Sort, Insertion Sort, BST Search, Linear Search, benchmarking, edit song, and load CSV.

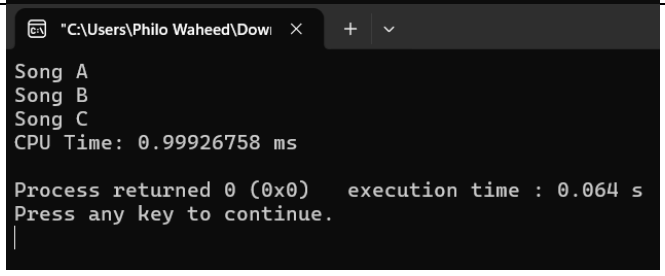
Overall, the new code is suitable for a clean Data Structures project, especially for demonstrating STL containers, sorting, searching, stack and queue usage, file handling, and menu-based console applications.

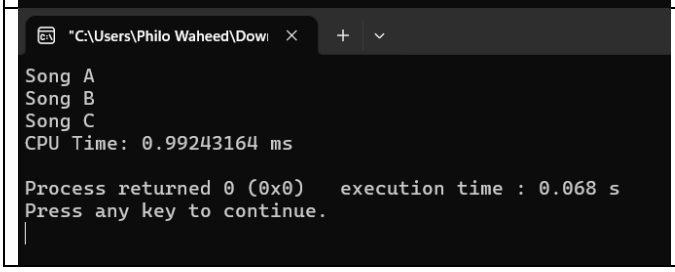
Quick Sorting _ CPU Timing Test

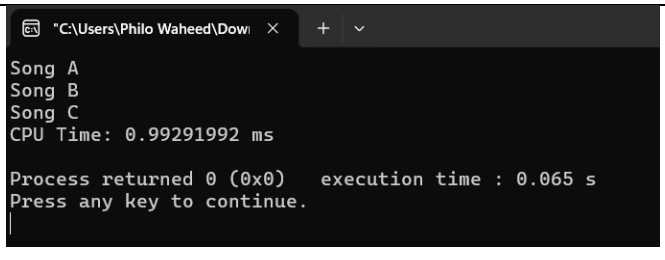


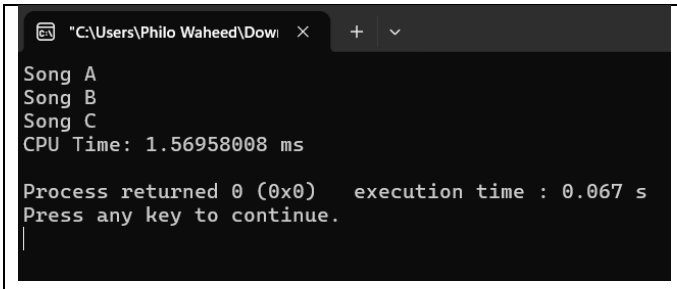


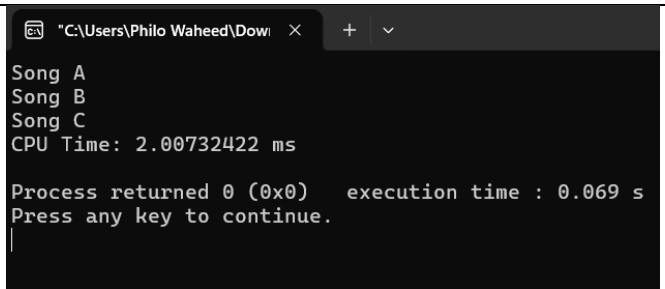


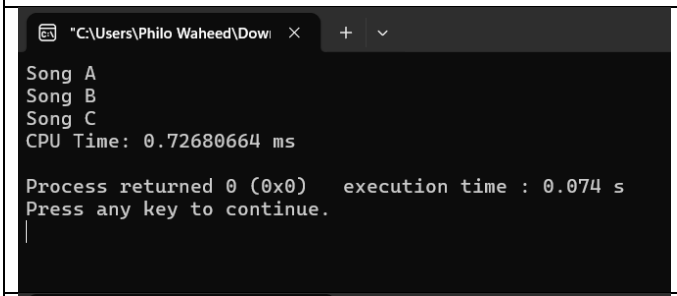


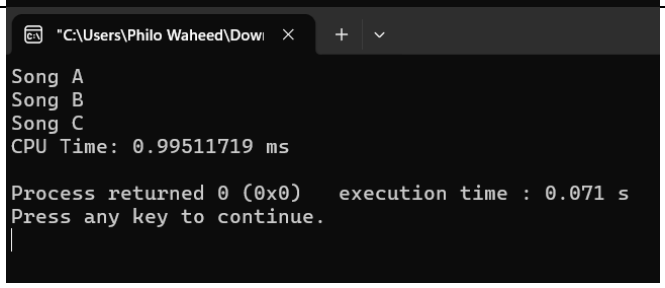


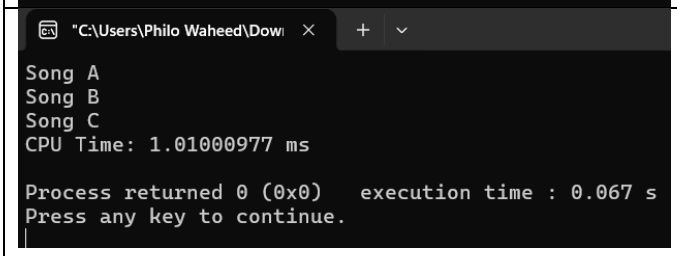


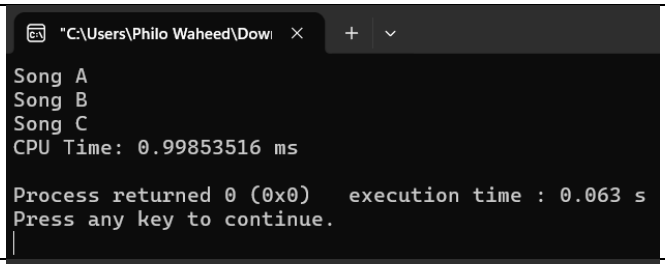

"C:\Users\Philo Waheed\Dow
Song A
Song B
Song C
CPU Time: 1.56958008 ms
Process returned 0 (0x0) execution time : 0.067 s
Press any key to continue.

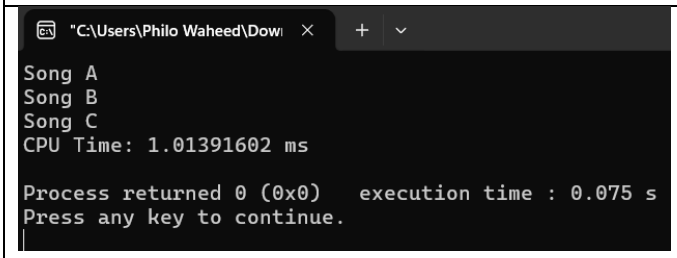

"C:\Users\Philo Waheed\Dow
Song A
Song B
Song C
CPU Time: 2.00732422 ms
Process returned 0 (0x0) execution time : 0.069 s
Press any key to continue.

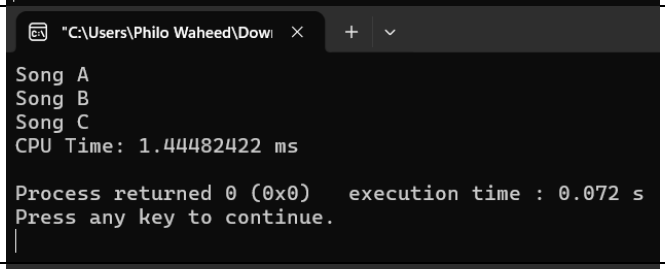

"C:\Users\Philo Waheed\Dow
Song A
Song B
Song C
CPU Time: 0.72680664 ms
Process returned 0 (0x0) execution time : 0.074 s
Press any key to continue.

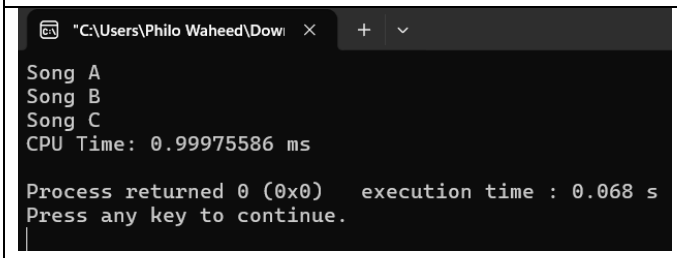

"C:\Users\Philo Waheed\Dow
Song A
Song B
Song C
CPU Time: 0.99511719 ms
Process returned 0 (0x0) execution time : 0.071 s
Press any key to continue.

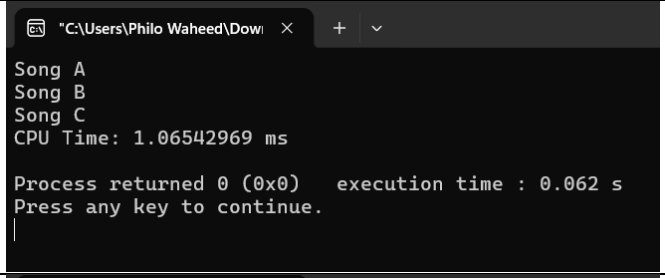

"C:\Users\Philo Waheed\Dow
Song A
Song B
Song C
CPU Time: 1.01000977 ms
Process returned 0 (0x0) execution time : 0.067 s
Press any key to continue.

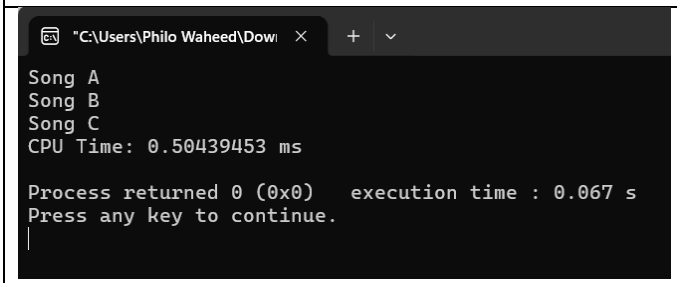

"C:\Users\Philo Waheed\Dow
Song A
Song B
Song C
CPU Time: 0.99853516 ms
Process returned 0 (0x0) execution time : 0.063 s
Press any key to continue.

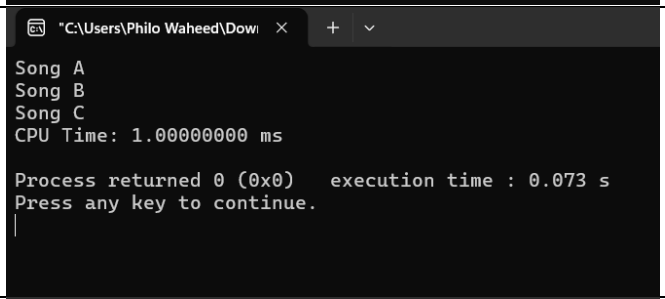

"C:\Users\Philo Waheed\Dow
Song A
Song B
Song C
CPU Time: 1.01391602 ms
Process returned 0 (0x0) execution time : 0.075 s
Press any key to continue.

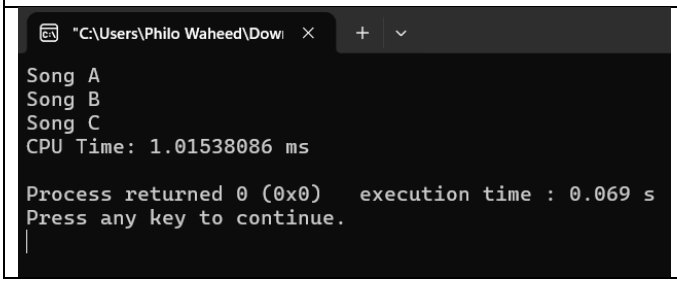

"C:\Users\Philo Waheed\Dow
Song A
Song B
Song C
CPU Time: 1.44482422 ms
Process returned 0 (0x0) execution time : 0.072 s
Press any key to continue.

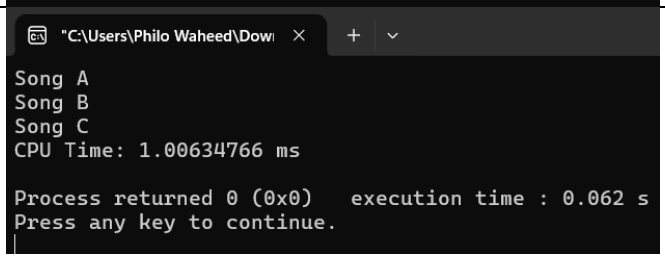

"C:\Users\Philo Waheed\Dow
Song A
Song B
Song C
CPU Time: 0.99975586 ms
Process returned 0 (0x0) execution time : 0.068 s
Press any key to continue.

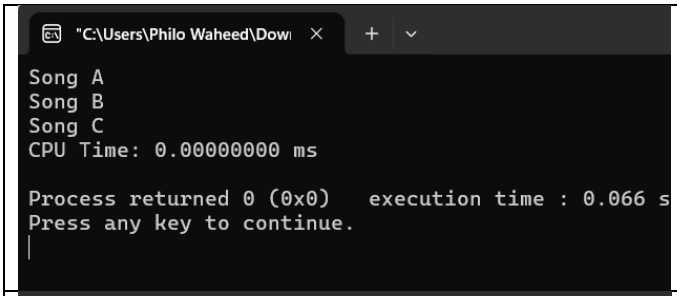

"C:\Users\Philo Waheed\Dow
Song A
Song B
Song C
CPU Time: 1.06542969 ms
Process returned 0 (0x0) execution time : 0.062 s
Press any key to continue.

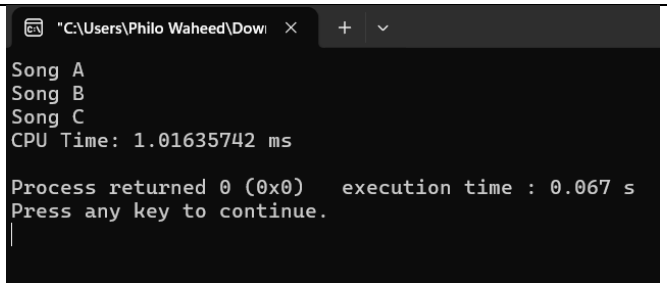

"C:\Users\Philo Waheed\Dow
Song A
Song B
Song C
CPU Time: 0.50439453 ms
Process returned 0 (0x0) execution time : 0.067 s
Press any key to continue.

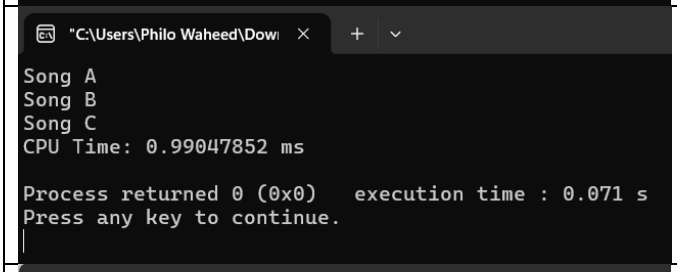

"C:\Users\Philo Waheed\Dow
Song A
Song B
Song C
CPU Time: 1.00000000 ms
Process returned 0 (0x0) execution time : 0.073 s
Press any key to continue.

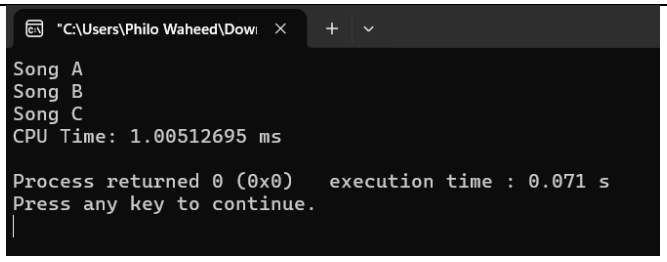

"C:\Users\Philo Waheed\Dow
Song A
Song B
Song C
CPU Time: 1.01538086 ms
Process returned 0 (0x0) execution time : 0.069 s
Press any key to continue.

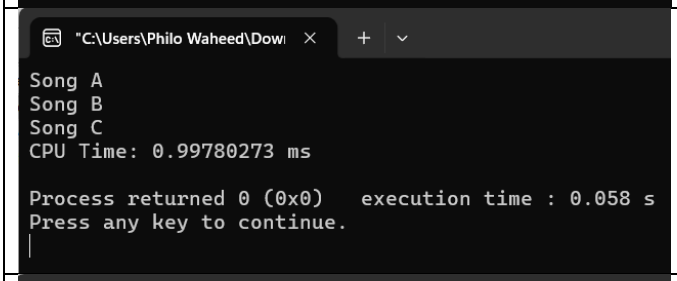

"C:\Users\Philo Waheed\Dow
Song A
Song B
Song C
CPU Time: 1.00634766 ms
Process returned 0 (0x0) execution time : 0.062 s
Press any key to continue.

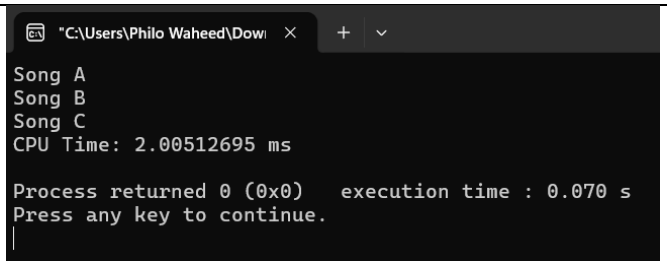


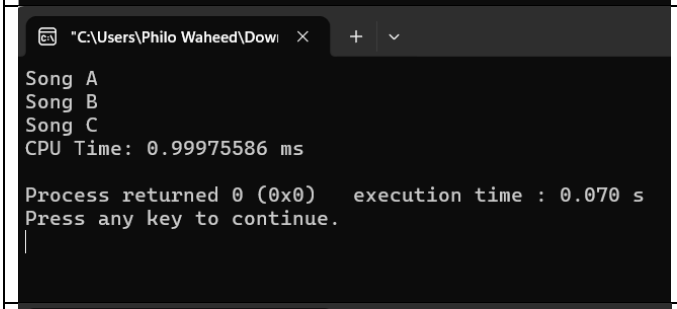


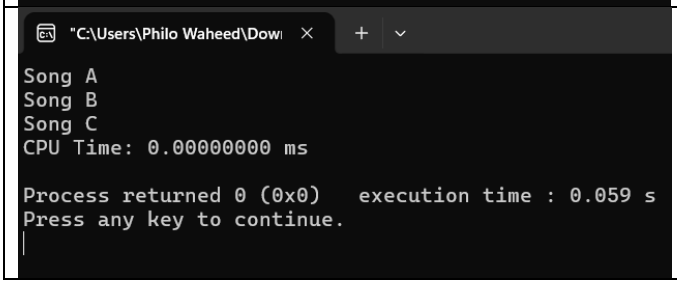


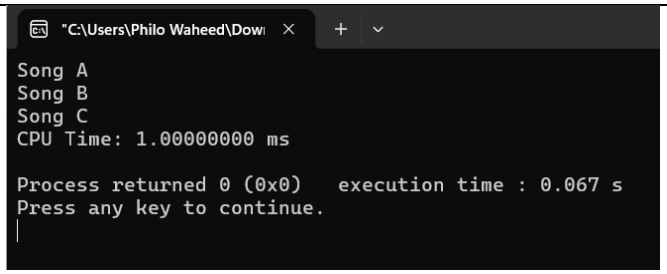




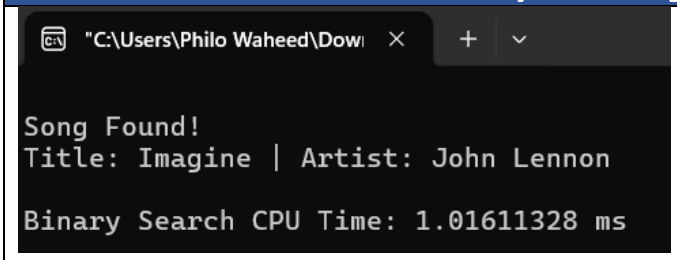


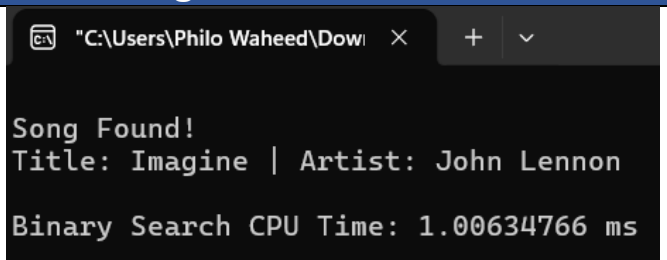


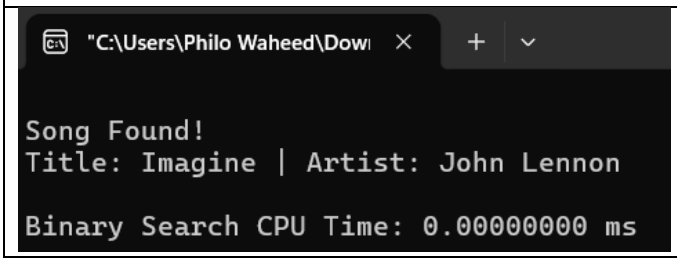


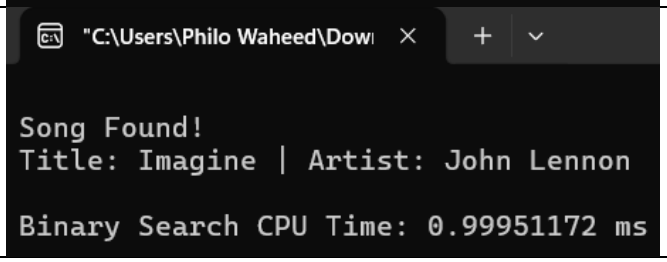


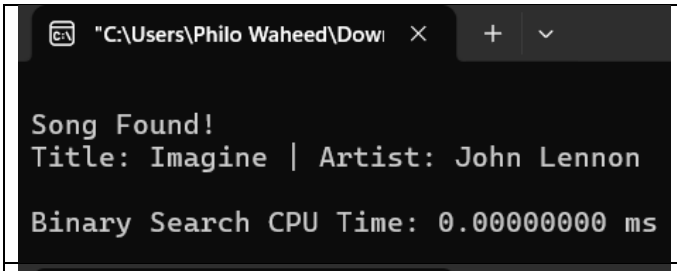
Binary Search _ CPU Timing Test

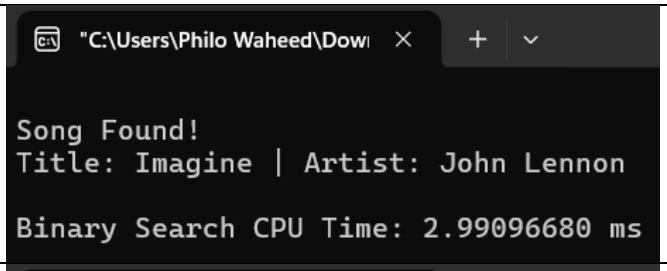


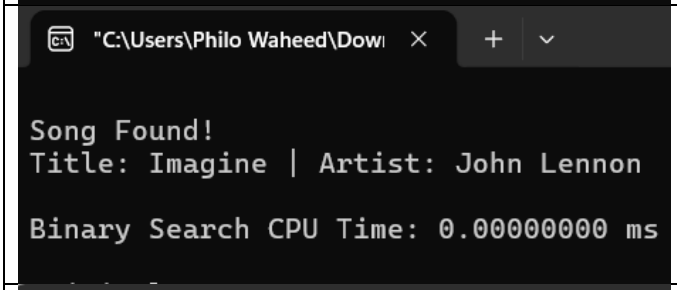


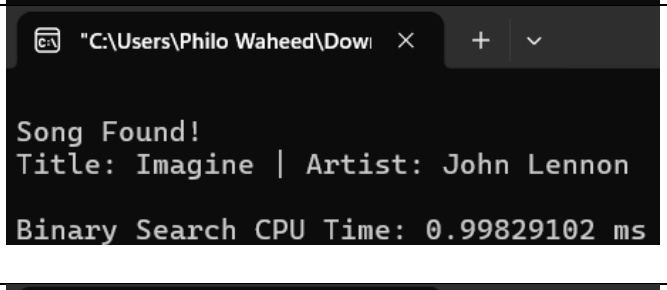


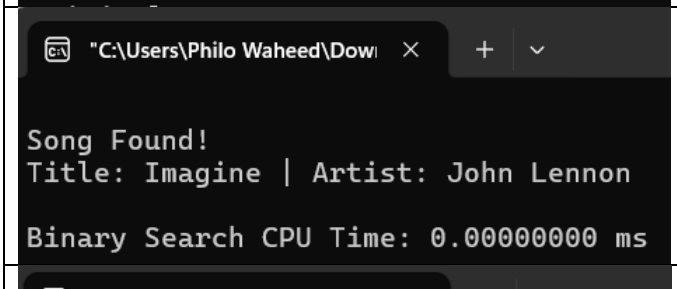


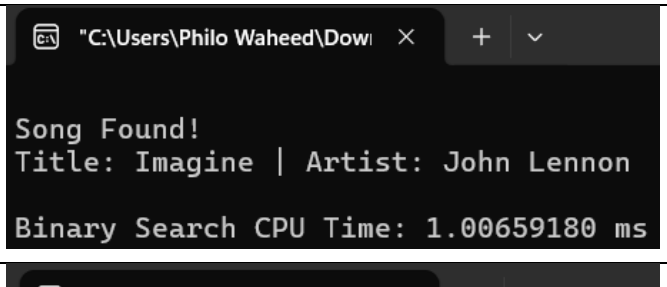

"C:\Users\Philo Waheed\Dow
Song Found!
Title: Imagine | Artist: John Lennon
Binary Search CPU Time: 0.00000000 ms

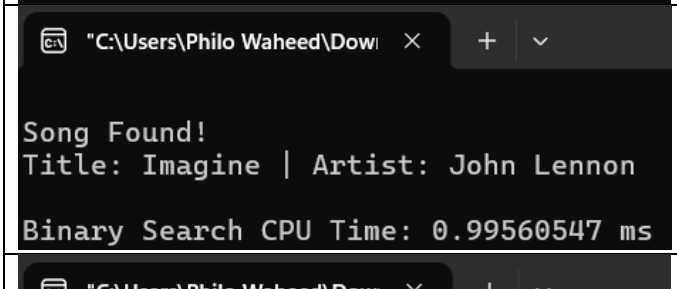

"C:\Users\Philo Waheed\Dow
Song Found!
Title: Imagine | Artist: John Lennon
Binary Search CPU Time: 2.99096680 ms

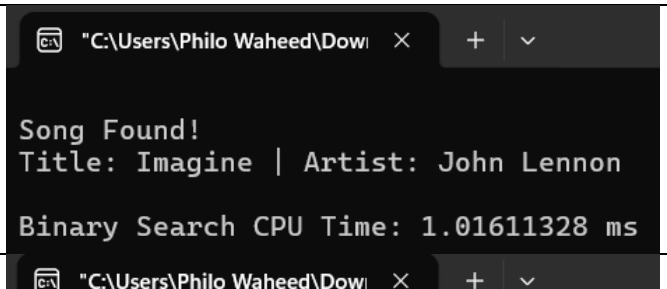

"C:\Users\Philo Waheed\Dow
Song Found!
Title: Imagine | Artist: John Lennon
Binary Search CPU Time: 0.00000000 ms

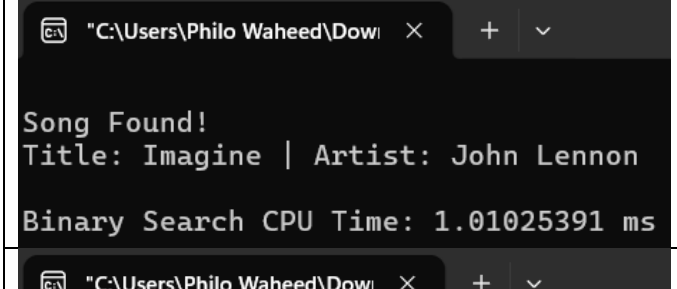

"C:\Users\Philo Waheed\Dow
Song Found!
Title: Imagine | Artist: John Lennon
Binary Search CPU Time: 0.99829102 ms

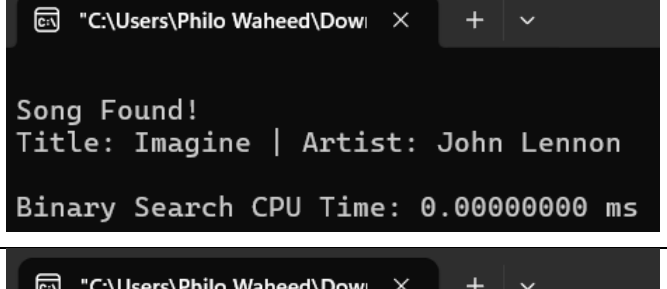

"C:\Users\Philo Waheed\Dow
Song Found!
Title: Imagine | Artist: John Lennon
Binary Search CPU Time: 0.00000000 ms

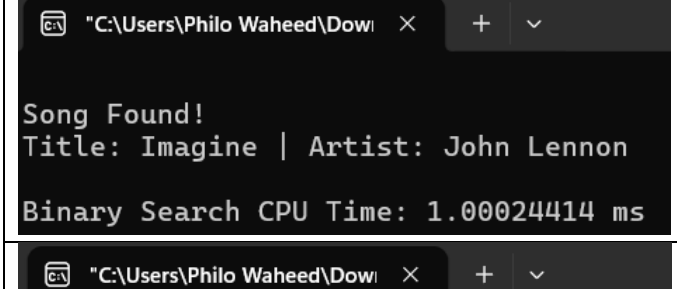

"C:\Users\Philo Waheed\Dow
Song Found!
Title: Imagine | Artist: John Lennon
Binary Search CPU Time: 1.00659180 ms


"C:\Users\Philo Waheed\Dow
Song Found!
Title: Imagine | Artist: John Lennon
Binary Search CPU Time: 0.99560547 ms

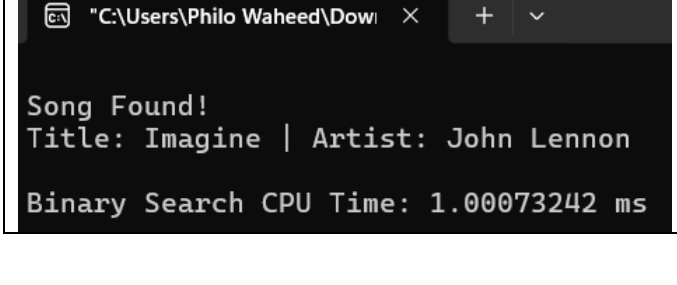

"C:\Users\Philo Waheed\Dow
Song Found!
Title: Imagine | Artist: John Lennon
Binary Search CPU Time: 1.01611328 ms

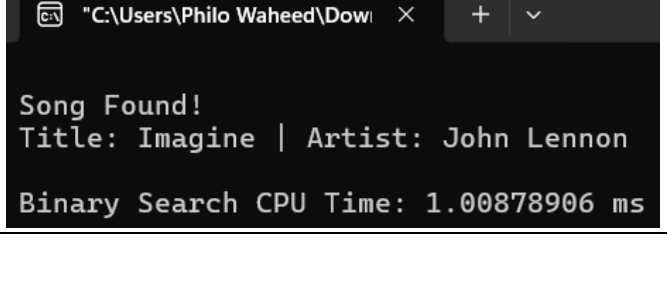

"C:\Users\Philo Waheed\Dow
Song Found!
Title: Imagine | Artist: John Lennon
Binary Search CPU Time: 1.01025391 ms

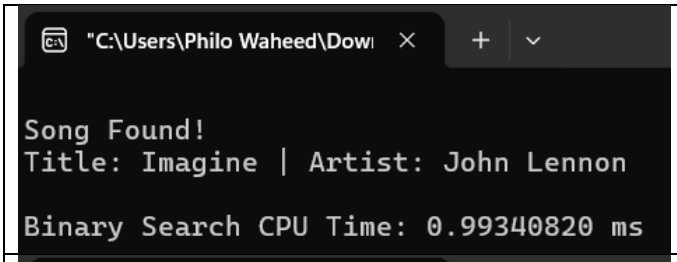

"C:\Users\Philo Waheed\Dow
Song Found!
Title: Imagine | Artist: John Lennon
Binary Search CPU Time: 0.00000000 ms

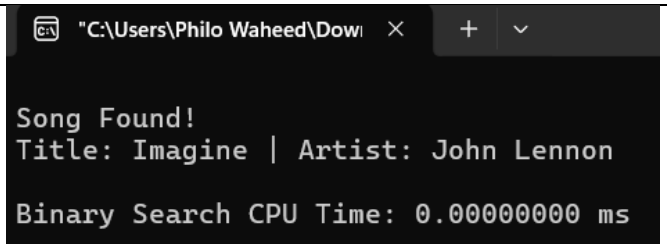

"C:\Users\Philo Waheed\Dow
Song Found!
Title: Imagine | Artist: John Lennon
Binary Search CPU Time: 1.00024414 ms


"C:\Users\Philo Waheed\Dow
Song Found!
Title: Imagine | Artist: John Lennon
Binary Search CPU Time: 2.70996094 ms

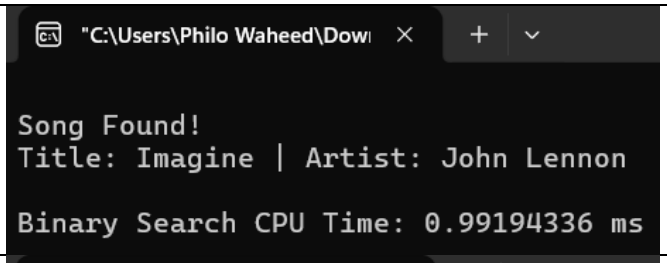

"C:\Users\Philo Waheed\Dow
Song Found!
Title: Imagine | Artist: John Lennon
Binary Search CPU Time: 1.00073242 ms

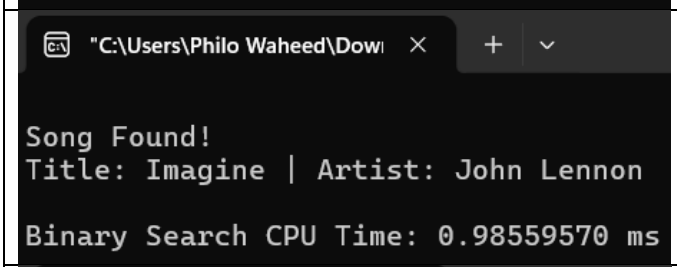

"C:\Users\Philo Waheed\Dow
Song Found!
Title: Imagine | Artist: John Lennon
Binary Search CPU Time: 1.00878906 ms

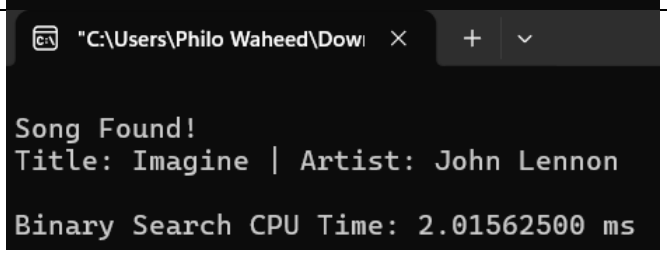

"C:\Users\Philo Waheed\Dow
Song Found!
Title: Imagine | Artist: John Lennon
Binary Search CPU Time: 0.99340820 ms

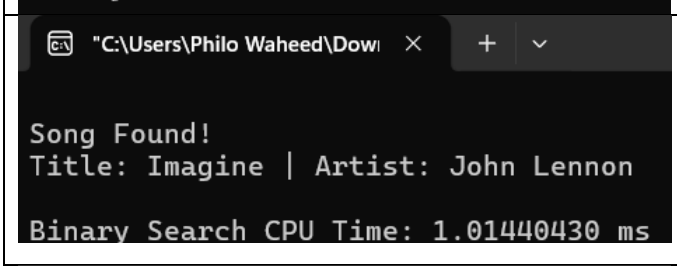

"C:\Users\Philo Waheed\Dow
Song Found!
Title: Imagine | Artist: John Lennon
Binary Search CPU Time: 0.00000000 ms

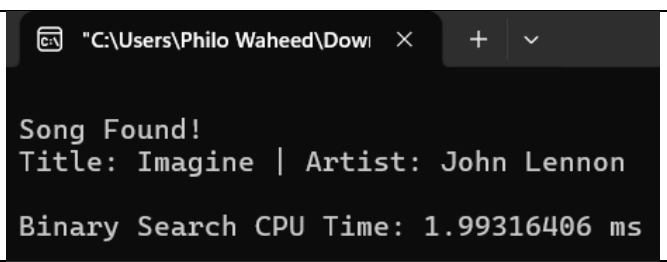

"C:\Users\Philo Waheed\Dow
Song Found!
Title: Imagine | Artist: John Lennon
Binary Search CPU Time: 0.99951172 ms

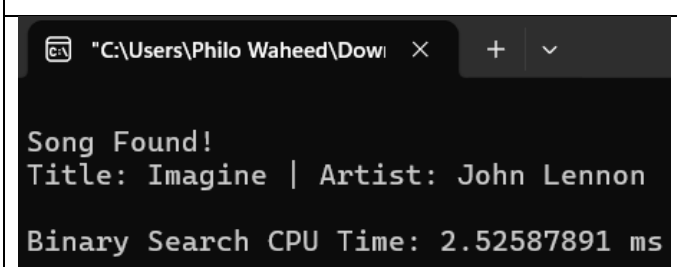

"C:\Users\Philo Waheed\Dow
Song Found!
Title: Imagine | Artist: John Lennon
Binary Search CPU Time: 0.99194336 ms

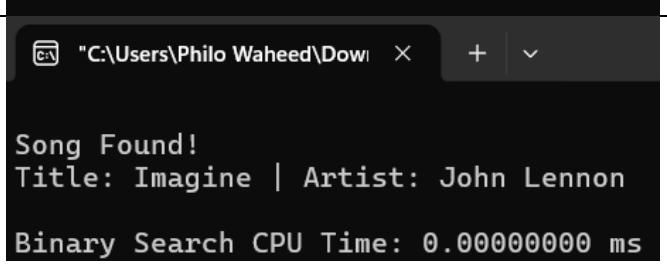

"C:\Users\Philo Waheed\Dow
Song Found!
Title: Imagine | Artist: John Lennon
Binary Search CPU Time: 0.98559570 ms

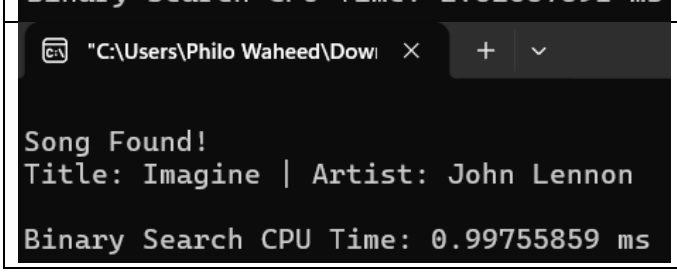

"C:\Users\Philo Waheed\Dow
Song Found!
Title: Imagine | Artist: John Lennon
Binary Search CPU Time: 2.01562500 ms

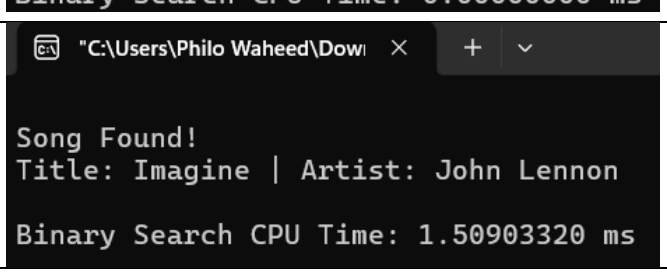

"C:\Users\Philo Waheed\Dow
Song Found!
Title: Imagine | Artist: John Lennon
Binary Search CPU Time: 1.01440430 ms


"C:\Users\Philo Waheed\Dow
Song Found!
Title: Imagine | Artist: John Lennon
Binary Search CPU Time: 1.99316406 ms


"C:\Users\Philo Waheed\Dow
Song Found!
Title: Imagine | Artist: John Lennon
Binary Search CPU Time: 2.52587891 ms


"C:\Users\Philo Waheed\Dow
Song Found!
Title: Imagine | Artist: John Lennon
Binary Search CPU Time: 0.00000000 ms


"C:\Users\Philo Waheed\Dow
Song Found!
Title: Imagine | Artist: John Lennon
Binary Search CPU Time: 0.99755859 ms


"C:\Users\Philo Waheed\Dow
Song Found!
Title: Imagine | Artist: John Lennon
Binary Search CPU Time: 1.50903320 ms